

ASSIGNMENT-1 Artificial Intelligence

MANVENDRA PARATP SINGH(G24AIT2163)

INDIAN INSTITUTE OF TECHNOLOGY-JODHPUR

Course Number - CSL7XX0

Instructor - Dr. Debasis Das

Date 23-MARCH-2025



Q A.) Dynamic Goal-Based Agent for Warehouse Logistics Optimization.

A robotic agent operates in a warehouse modeled as an $N \times M$ grid environment. The agent starts at a predefined loading dock and must deliver packages to multiple destinations marked on the grid while avoiding dynamically placed obstacles.

Take suitable values of the following parameters.

- Warehouse dimensions: $N \times M$ grid size (M, N between 5 and 10, inclusive)
- Number of packages: P (between 2 and 6, inclusive)
- Number of obstacles: O (between 1 and 10, inclusive)
- Package locations: $(X_1, Y_1), (X_2, Y_2), \dots (X_P, Y_P)$
- Drop-off locations: $(D_1X, D_1Y), (D_2X, D_2Y), \dots (D_{PX}, D_{PY})$
- Robot starting position: $S=(S_x, S_y)$, starts at a fixed cell but moves dynamically
- Movement cost: Each movement incurs a cost of 1 unit
- Delivery reward: Successfully delivering a package adds 10 units to the total reward
- Obstacle penalty: Hitting an obstacle results in a (-5) penalty

Note : Package locations and drop-off locations should not overlap.

Q1. Represent the warehouse as an $N \times M$ matrix. Place the packages, drop-off points, and obstacles randomly. Display the initial warehouse configuration.

Solution:

We consider a fixed **8×8** warehouse matrix with **5 packages** to be delivered and **6 obstacles** placed randomly and **robot position** is at (0,0) . The initial warehouse configuration is displayed accordingly.

The implementation is available in the [.ipynb](#) file (Q1).

Here is the output of that.

Initial Warehouse Setup:

```
R X P D . . . .
X X . . . . . .
. . . . . . . .
. . X P . D . .
D . P X . . . .
. . . . . P . .
X . . P . D . .
. . . . D . . .
```

Q2. Implement a goal-based agent that can identify all goals, plan a sequence of actions to reach the goal, use a search algorithm (BFS, DFS, or UCS) to find optimal paths, deliver all packages, and calculate the total cost.

Solution:

Breadth-First Search (**BFS**) is selected as the search algorithm for this grid-based problem due to the **uniform cost associated** with all grid movements. BFS efficiently identifies the optimal path by systematically exploring nodes level by level, ensuring the shortest path is found.

While Uniform Cost Search (UCS) is suitable for weighted graphs, its computational overhead is deemed unnecessary in this scenario where edge costs are uniform.

Although Depth-First Search (DFS) could be considered, it does not inherently guarantee the shortest path. Therefore, BFS emerges as the most appropriate choice for planning the sequence of actions for package delivery and calculating the total cost.

The implementation is available in the [.ipynb](#) file (Q2).

Here is the output of that.

Packages to be delivered: 5

Robot starts from (0, 0) to pick up the package at (3, 3).

Path taken for pickup: [(0, 0), (1, 0), (2, 0), (3, 0), (3, 1), (3, 2), (3, 3)]

Robot needs to deliver the package from (3, 3) to (0, 3).

Path taken for delivery: [(3, 3), (2, 3), (1, 3), (0, 3)]

Robot starts from (0, 3) to pick up the package at (6, 3).

Path taken for pickup: [(0, 3), (1, 3), (2, 3), (3, 3), (4, 3), (5, 3), (6, 3)]

Robot needs to deliver the package from (6, 3) to (7, 4).

Path taken for delivery: [(6, 3), (7, 3), (7, 4)]

Robot starts from (7, 4) to pick up the package at (0, 2).

Path taken for pickup: [(7, 4), (6, 4), (5, 4), (4, 4), (3, 4), (2, 4), (1, 4), (0, 4), (0, 3), (0, 2)]

Robot needs to deliver the package from (0, 2) to (6, 5).

Path taken for delivery: [(0, 2), (1, 2), (2, 2), (3, 2), (4, 2), (5, 2), (6, 2), (6, 3), (6, 4), (6, 5)]

Robot starts from (6, 5) to pick up the package at (4, 2).

Path taken for pickup: [(6, 5), (5, 5), (4, 5), (4, 4), (4, 3), (4, 2)]

Robot needs to deliver the package from (4, 2) to (3, 5).

Path taken for delivery: [(4, 2), (3, 2), (3, 3), (3, 4), (3, 5)]

Robot starts from (3, 5) to pick up the package at (5, 5).

Path taken for pickup: [(3, 5), (4, 5), (5, 5)]

Robot needs to deliver the package from (5, 5) to (4, 0).

Path taken for delivery: [(5, 5), (4, 5), (4, 4), (4, 3), (4, 2), (4, 1), (4, 0)]

Total Movement Cost: 52

Total Delivery Reward: 50

Obstacle Penalty: -35

Final Score: -37

Q3. Choose a random seed value for the ease of reproducing the results. Your program should give outputs: the chosen path taken by the agent, total cost and rewards, final score based on penalties, movement costs, and successful deliveries.

Solution:

Reproducibility is ensured by using a fixed random seed, guaranteeing consistent outcomes across multiple runs. The program now supports dynamic inputs, allowing flexibility in defining warehouse dimensions, package locations, obstacles, and the agent's initial position. Here are the updated values:

```
Now warehouse values changed from static to random values
Here are the updated values
```

```
Warehouse grid -> 10*8 grid
Package -> 4
Obstacles -> 5
Robot position -> (2, 3)
```

The implementation is available in the [.ipynb](#) file (Q3).

Here is the output of that.

Updated warehouse after random values

```
Initial Warehouse Setup:
```

```
P . D . . . . .
X X . D . . . .
. . . R . . . .
P . X . . . . D
. . . X . . . .
. . . . . . . .
. . . . . . . .
. . . . . . . .
. . . D . . P .
. . . . P . X .
```


The packages delivered output is attached here:

Packages to be delivered: 6

✓ Robot starts from (6, 1) to pick up the package at (3, 2).
Path taken for pickup: [(6, 1), (5, 1), (4, 1), (3, 1), (3, 2)]
Robot needs to deliver the package from (3, 2) to (1, 1).
Path taken for delivery: [(3, 2), (2, 2), (1, 2), (1, 1)]

Robot starts from (1, 1) to pick up the package at (9, 6).
Path taken for pickup: [(1, 1), (2, 1), (3, 1), (4, 1), (5, 1), (6, 1), (7, 1), (8, 1), (9, 1), (9, 2), (9, 3), (9, 4), (9, 5), (9, 6)]
Robot needs to deliver the package from (9, 6) to (0, 0).
Path taken for delivery: [(9, 6), (8, 6), (7, 6), (6, 6), (5, 6), (4, 6), (3, 6), (2, 6), (1, 6), (0, 6), (0, 5), (0, 4), (0, 3), (0, 2), (0, 1), (0, 0)]

Robot starts from (0, 0) to pick up the package at (1, 3).
Path taken for pickup: [(0, 0), (1, 0), (1, 1), (1, 2), (1, 3)]
Robot needs to deliver the package from (1, 3) to (3, 0).
Path taken for delivery: [(1, 3), (2, 3), (3, 3), (3, 2), (3, 1), (3, 0)]

Robot starts from (3, 0) to pick up the package at (8, 3).
Path taken for pickup: [(3, 0), (4, 0), (5, 0), (6, 0), (7, 0), (8, 0), (8, 1), (8, 2), (8, 3)]
Robot needs to deliver the package from (8, 3) to (8, 6).
Path taken for delivery: [(8, 3), (8, 4), (8, 5), (8, 6)]

Robot starts from (8, 6) to pick up the package at (3, 7).
Path taken for pickup: [(8, 6), (7, 6), (6, 6), (5, 6), (4, 6), (3, 6), (3, 7)]
Robot needs to deliver the package from (3, 7) to (9, 4).
Path taken for delivery: [(3, 7), (4, 7), (5, 7), (6, 7), (7, 7), (8, 7), (9, 7), (9, 6), (9, 5), (9, 4)]

Robot starts from (9, 4) to pick up the package at (0, 2).
Path taken for pickup: [(9, 4), (8, 4), (7, 4), (6, 4), (5, 4), (4, 4), (3, 4), (2, 4), (1, 4), (0, 4), (0, 3), (0, 2)]
Robot needs to deliver the package from (0, 2) to (6, 5).
Path taken for delivery: [(0, 2), (1, 2), (2, 2), (3, 2), (4, 2), (5, 2), (6, 2), (6, 3), (6, 4), (6, 5)]

Total Movement Cost: 90
Total Delivery Reward: 60
Obstacle Penalty: -5

Final Score: -35

Q B) Suppose you and your friend live in different cities on a map and you both are planning for a common meetup place which is optimal for you both. Let's find the common meetup with a little twist.

On every turn, you and your friend can simultaneously move to a neighboring city on the map. The amount of time needed to move from city i to neighbor j is equal to the straight line distance $d(i, j) \times 2$ between the cities, but on each turn the friend that arrives first must wait until the other one arrives (and calls the first on his/her cell phone) before the next turn can begin. The heuristic you are assuming is the straight line distance $d(i, j)$. You both friends want to meet as quickly as possible.

Q1. Formulate this search problem and display the map of the city with heuristic (h) and transition cost (g).

Solution:

To formulate this search problem, we first identify the **neighboring cities** for both you and your friend. Each city acts as a **node**, and the direct connections between them represent **edges**.

1. **Heuristic Function (h):** The heuristic value for each city is the **straight-line distance** to your friend's location, representing an estimated cost to reach the meetup point.
2. **Transition Cost (g):** The movement cost between two cities is **$d(i, j) \times 2$** , where **$d(i, j)$** is the straight-line distance between connected cities.
3. **Graph Representation:** The city map is displayed with **nodes** (cities), **edges** (connections), and labels for heuristic values and transition costs.

By computing heuristic values from neighboring cities to your friend's location, we ensure an informed search strategy for finding the optimal meeting point.

The implementation is available in the [.ipynb](#) file(Q1), and the output screenshot is attached below.

```
City 1: Naini Tal & City 2: Indore
City 1: 29.317087269390218, 79.45483872140413
City 2: 22.71663343073364, 75.78138525626387
City 1: Naini Tal & City 2: Indore are 818.35 km apart.
```

Neighboring cities within the range is : 68
Neighboring cities within the range is : 193
Neighboring cities within the range is : 56
City : Naini Tal --240.38966855854113-->Delhi
City : Naini Tal --346.96987121383-->Bhiwani
City : Naini Tal --244.30593418221898-->Faridabad
City : Naini Tal --272.89645124140856-->Gurgaon
City : Naini Tal --354.8842589523224-->Hisar
City : Naini Tal --288.0955608621943-->Jhajjar
City : Naini Tal --347.82221093206397-->Mahendragarh
City : Naini Tal --246.65683204314234-->Panipat
City : Naini Tal --308.5769012218975-->Rewari
City : Naini Tal --289.96812750245306-->Rohtak
City : Naini Tal --253.50346502037272-->Sonapat

City : Indore --185.4764208742048-->Bhopal
City : Indore --401.48576082302566-->Damoh
City : Indore --449.8274731074069-->Datia
City : Indore --69.21212157150936-->Dewas
City : Indore --249.95872845546947-->Guna
City : Indore --439.4154590880448-->Gwalior
City : Indore --0.04017563461104693-->Indore
City : Indore --461.03822535737373-->Morena
City : Indore --339.62046193197426-->Narsinghpur
City : Indore --245.55951899540275-->Raisen
City : Indore --160.07562495812607-->Rajgarh
City : Indore --328.9536890769425-->Sagar
City : Indore --141.30667004769384-->Sehore
City : Indore --111.92169416553405-->Shajapur

Q2. Implement search strategies: a. Greedy Best First Search b. A* and provide the Search Cost (nodes generated, space and time taken for execution) for both of them.

Solution:

We implemented **Greedy Best-First Search (GBFS)** and **A*** to find the optimal meeting point.

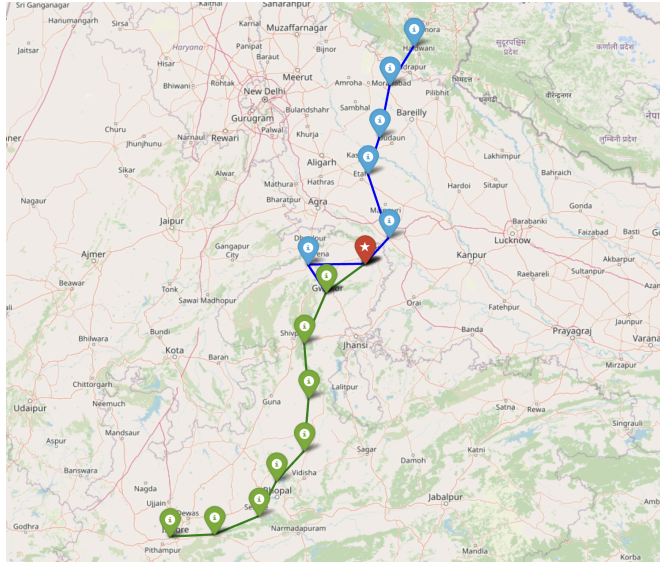
- **Greedy Best-First Search (GBFS):** Uses only the **heuristic (h)**, prioritizing cities closest to the goal. It's fast but may get stuck in suboptimal paths.
- **A*:** Uses both **heuristic (h) and actual cost (g)**, ensuring an optimal path but taking more time and space.

Output HTML file is also attached *gbfs_straight_line.html* and *a_start_straight_line.html*

The implementation is available in the *.ipynb* file, and the output screenshot is attached below.

GBFS

```
GBFS Result with Road enable = False
My City City: Naini Tal & Friend City: Indore
Meeting City: Bhind
Total Cost: 1272.2992860542645
Nodes Generated: 177
Max Frontier Size: 170
Execution Time: 0.0011 sec
Path: ['Naini Tal-Indore', 'Rampur-Dewas', 'Badaun-Sehore', 'Etah-B
My Path: ['Naini Tal', 'Rampur', 'Badaun', 'Etah', 'Etawah', 'Bhind
Friend Path: ['Indore', 'Dewas', 'Sehore', 'Bhopal', 'Vidisha', 'As
Meeting points: (26.426501123712743, 78.71485018127507)
```



A*

A* Result with Road enable = False

My City City: Naini Tal & Friend City: Indore

Meeting City: Gwalior

Total Cost: 1029.454798205413

Nodes Generated: 16935

Max Frontier Size: 8176

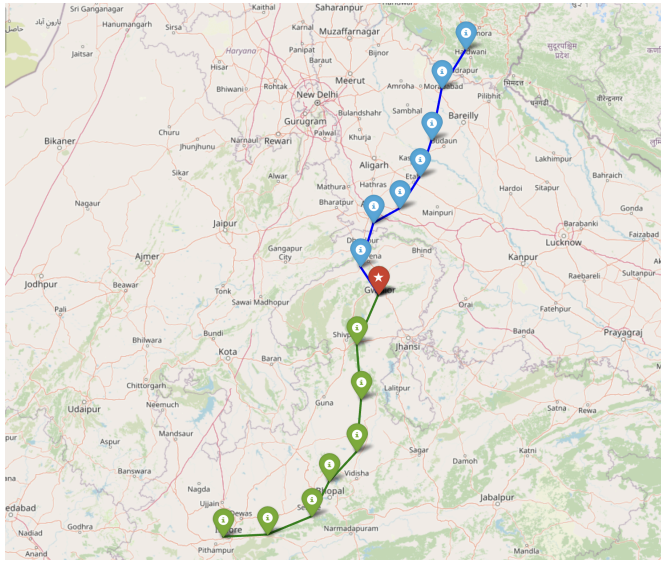
Execution Time: 0.4099 sec

Path: ['Naini Tal-Indore', 'Rampur-Dewas', 'Badaun-Sehore', 'Eta

My Path: ['Naini Tal', 'Rampur', 'Badaun', 'Etah', 'Firozabad',

Friend Path: ['Indore', 'Dewas', 'Sehore', 'Bhopal', 'Vidisha',

Meeting points: (26.045045627381565, 78.13879230963495)



Q3. Change the heuristic function $d(i, j)$ from the straight-line distance between cities i and j to more realistic functions such as road or train route distance. And provide the analysis of the solution for different heuristic functions in terms of Search Cost (nodes generated, space and time taken for execution).

Solution:

We modified the heuristic function from **straight-line distance (using latitude and longitude)** to **real-world road distances** between cities. This makes the search process more realistic as it accounts for actual travel paths rather than idealized straight-line movement

The implementation is available in the [.ipynb](#) file, and the output screenshot is attached below.

Output HTML file is also attached [gbfs_by_road.html](#) and [a_start_by_road.html](#)

BFS

GBFS Result with Road enable = True

My City City: Naini Tal & Friend City: Indore

Meeting City: Bhind

Total Cost: 1272.2992860542645

Nodes Generated: 177

Max Frontier Size: 170

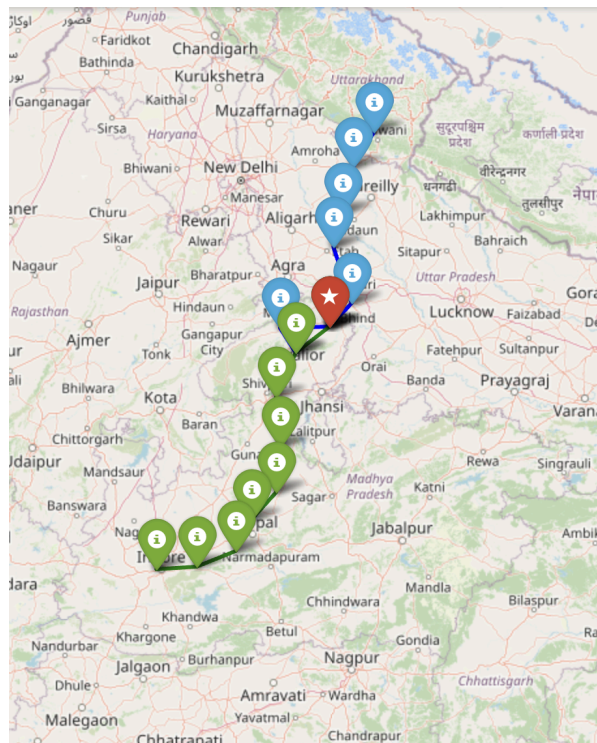
Execution Time: 0.0007 sec

Path: ['Naini Tal-Indore', 'Rampur-Dewas', 'Badaun-Sehore', 'Bhind', 'Indore']

My Path: ['Naini Tal', 'Rampur', 'Badaun', 'Etah', 'Etawah', 'Bhind', 'Indore']

Friend Path: ['Indore', 'Dewas', 'Sehore', 'Bhopal', 'Vidisha', 'Bhind', 'Indore']

Meeting points: (26.426501123712743, 78.71485018127507)



A*

A* Result with Road enable = True

My City City: Naini Tal & Friend City: Indore

Meeting City: Gwalior

Total Cost: 1029.454798205413

Nodes Generated: 16935

Max Frontier Size: 8176

Execution Time: 0.0966 sec

Path: ['Naini Tal-Indore', 'Rampur-Dewas', 'Badaun-Sehore', 'Etah-Bhopal', 'Fi

My Path: ['Naini Tal', 'Rampur', 'Badaun', 'Etah', 'Firozabad', 'Agra', 'Morer

Friend Path: ['Indore', 'Dewas', 'Sehore', 'Bhopal', 'Vidisha', 'Ashoknagar',

Meeting points: (26.045045627381565, 78.13879230963495)

